

R308 - TP1

Objectifs

- Implémenter des classes et instancier des objets.
- Distinguer et utiliser les attributs d'instance et de classe.
- Définir des méthodes d'instance pour manipuler l'état d'un objet.
- Utiliser les propriétés (`@property`) pour l'encapsulation.
- Mettre en œuvre des constructeurs alternatifs (`@classmethod`).
- Utiliser les méthodes spéciales (`__repr__` , `__eq__`) pour un comportement d'objet plus riche.
- Intégrer ces concepts dans un mini-projet.

Partie 1 : Prise en Main (60 minutes)

Exercice 1 : La classe Point2D (30 min)

Objectifs : Maîtriser la syntaxe de base : `class` , `__init__` , `self` , attributs d'instance, méthodes d'instance.

Énoncé :

Créez un fichier `point.py` . Dans ce fichier, implémentez une classe Point2D pour représenter un point dans un plan cartésien.

1. Le constructeur `__init__(self, x, y)` initialisera les attributs d'instance `x` et `y`.
2. La méthode `__repr__(self)` devra retourner une chaîne de caractères "officielle" et non ambiguë, par exemple `Point2D(x=1, y=2)`.
3. La méthode `deplacer(self, dx, dy)` modifiera les coordonnées du point en lui ajoutant `dx` et `dy`.
4. La méthode `distance(self, autre_point)` calculera et retournera la distance euclidienne entre le point courant (`self`) et une autre instance de Point2D. La formule est $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Vous aurez besoin d'importer le module `math`.

Code de départ (`point.py`) :

```

import math

class Point2D:
    """Représente un point dans un plan 2D."""

    def __init__(self, x: float = 0.0, y: float = 0.0):
        # TODO: initialiser x et y
        pass

    def __repr__(self) -> str:
        # TODO: retourner une représentation textuelle
        return ""

    def deplacer(self, dx: float, dy: float) -> None:
        # TODO: modifier les coordonnées x et y
        pass

    def distance(self, autre_point: "Point2D") -> float:
        # TODO: calculer et retourner la distance
        return 0.0

# --- Tests à valider ---
p1 = Point2D(1, 2)
p2 = Point2D(4, 6)

print(f"Point 1 : {p1}")
print(f"Point 2 : {p2}")

# Test de la distance
dist = p1.distance(p2)
print(f"Distance entre p1 et p2 : {dist}")
assert math.isclose(dist, 5.0), "La distance devrait être 5.0"

# Test du déplacement
p1.deplacer(1, -1)
print(f"Point 1 après déplacement : {p1}")
assert p1.x == 2 and p1.y == 1, "Le point devrait être à (2, 1)"

print("Tests de l'exercice 1 passés avec succès !")

```

Solution :

exo1_r308.py

Exercice 2 : Compteur d'instances avec Ticket (30 min)

Objectifs : Comprendre le rôle et l'utilisation des attributs de classe.

Énoncé :

Créez un fichier `ticket.py`. Implémentez une classe `Ticket` qui modélise des tickets numérotés uniques.

1. Ajoutez un **attribut de classe** `compteur` initialisé à 0.
2. Dans le constructeur `__init__`, chaque nouvelle instance de `Ticket` doit se voir assigner un `id` unique. Cet id sera la valeur actuelle de `Ticket.compteur`.
3. Après avoir assigné l'ID, incrémentez l'attribut de classe compteur.
4. Implémentez la méthode `__repr__` pour qu'elle affiche l'ID du ticket, par exemple `Ticket(id=0)`.

Code de départ (`ticket.py`) :

```
class Ticket:
    """Un ticket avec un identifiant unique auto-incrémenté."""

    # TODO: Créer un attribut de classe pour le compteur

    def __init__(self):
        # TODO: Assigner l'ID à l'instance et incrémenter le compteur
        pass

    def __repr__(self) -> str:
        # TODO: Retourner la représentation du ticket
        return ""

# --- Tests à valider ---
ticket1 = Ticket()
ticket2 = Ticket()
ticket3 = Ticket()

print(ticket1)
print(ticket2)
print(ticket3)

assert ticket1.id == 0
assert ticket2.id == 1
assert ticket3.id == 2
assert Ticket.compteur == 3, "Le compteur de classe devrait être à 3"

print("Tests de l'exercice 2 passés avec succès !")
```

Solution :

[exo2_r308.py](#)

Partie 2 : Encapsulation et Méthodes Avancées (60 minutes)

Exercice 3 : Encapsulation avec Temperature (30 min)

Objectifs : Mettre en pratique l'encapsulation avec les décorateurs `@property` et `@*.setter`.

Énoncé :

Créez un fichier `temperature.py`. Reprenez la classe `Temperature` du cours.

1. Le constructeur prend une température en Celsius. Stockez cette valeur dans un attribut "privé" par convention, par exemple `_celsius`.
2. Créez une propriété `celsius` en lecture (`@property`) qui retourne `_celsius`.
3. Créez le `setter` associé (`@celsius.setter`) qui permet de modifier `_celsius`. Ce setter doit **lever une `ValueError`** si la température assignée est inférieure au zéro absolu (-273.15 °C).
4. Ajoutez une propriété **en lecture seule** `fahrenheit` qui calcule et retourne la température équivalente en Fahrenheit. Formule : $F = C \times \frac{9}{5} + 32$.
5. Ajoutez une propriété **en lecture seule** `kelvin`. Formule : $K = C + 273.15$.

Code de départ (`temperature.py`) :

```
class Temperature:
    """Représente une température et permet des conversions."""
    ZERO_ABSOLU_C = -273.15

    def __init__(self, celsius: float):
        # TODO: Utiliser le setter pour initialiser la valeur
        pass

    @property
    def celsius(self) -> float:
        # TODO: Getter pour _celsius
        return 0.0

    @celsius.setter
    def celsius(self, value: float):
        # TODO: Setter avec validation
        pass

    @property
    def fahrenheit(self) -> float:
        # TODO: Propriété en lecture seule
        return 0.0

    @property
```

```

def kelvin(self) -> float:
    # TODO: Propriété en lecture seule
    return 0.0

# --- Tests à valider ---
t = Temperature(20)
assert t.celsius == 20
assert t.fahrenheit == 68
assert t.kelvin == 293.15

t.celsius = -10
assert t.celsius == -10
assert t.kelvin == 263.15

# Test de la validation
try:
    t_invalide = Temperature(-300)
except ValueError as e:
    print(f"Exception attrapée avec succès : {e}")

try:
    t.celsius = -400
except ValueError as e:
    print(f"Exception attrapée avec succès : {e}")

print("Tests de l'exercice 3 passés avec succès !")

```

Solution :

exo3_r308.py

Exercice 4 : Constructeurs Alternatifs et Méthodes Spéciales avec Heure (30 min)

Objectifs : Implémenter un constructeur alternatif avec `@classmethod` et définir l'égalité logique avec `__eq__`.

Énoncé :

Créez un fichier `heure.py`. Implémentez une classe `Heure`.

1. Le constructeur `__init__(self, h, m, s)` initialise les heures, minutes et secondes. Assurez-vous que les valeurs sont valides (par exemple, 0-23 pour h, 0-59 pour m/s). Vous pouvez utiliser des `assert` ou lever une `ValueError`.

2. Implémentez une méthode de classe (`@classmethod`) nommée `from_secondes(cls, total_secondes)` qui crée une instance de `Heure` à partir d'un nombre total de secondes écoulées depuis minuit. (Utilisez `divmod` pour simplifier les calculs).
3. Implémentez `__repr__` pour un affichage clair (ex: `Heure(14, 30, 05)`).
4. Implémentez `__str__` pour un affichage plus lisible (ex: `14:30:05`).
5. Implémentez `__eq__(self, other)` pour comparer deux instances de `Heure` . Elles sont égales si leurs heures, minutes et secondes sont identiques.

Code de départ (`heure.py`) :

```
class Heure:
    """Représente une heure de la journée (h, m, s)."""

    def __init__(self, h: int, m: int, s: int):
        # TODO: Initialiser et valider les attributs
        pass

    @classmethod
    def from_secondes(cls, total_secondes: int) -> "Heure":
        # TODO: Calculer h, m, s et retourner une nouvelle instance `cls(...)`
        return cls(0, 0, 0)

    def __repr__(self) -> str:
        # TODO
        return ""

    def __str__(self) -> str:
        # TODO
        return ""

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Heure):
            return NotImplemented
        # TODO: Comparer self et other
        return False

# --- Tests à valider ---
h1 = Heure(1, 1, 1)
print(f"repr(h1): {repr(h1)}")
print(f"str(h1): {str(h1)}")

# Test du classmethod
total_secs = 3661 # 1 heure, 1 minute, 1 seconde
h2 = Heure.from_secondes(total_secs)
assert h2.h == 1 and h2.m == 1 and h2.s == 1

# Test de l'égalité
h3 = Heure(1, 1, 1)
```

```
assert h1 == h2
assert h1 == h3
assert h2 != Heure(1, 1, 2)

print("Tests de l'exercice 4 passés avec succès !")
```

exo4_R308.py

Partie 3 : Mini-Projet de Synthèse (60 minutes)

Exercice 5 : Gestion d'un Parc d'Équipements Réseau

Objectifs : Intégrer tous les concepts vus précédemment dans un projet cohérent et pertinent pour le domaine des Réseaux et Télécoms.

Contexte :

Vous devez développer un système simple pour gérer un parc d'équipements réseau (routeurs, switchs...). Vous créerez deux classes : `EquipementReseau` qui représente un appareil, et `GestionnaireParc` qui gère une collection d'appareils.

Énoncé :

1. Classe `EquipementReseau` (`equipement.py`)

- **Attribut de classe :** `compteur_id = 0` pour générer un ID unique pour chaque équipement.
- **Constructeur** `__init__(self, hostname, ip_address)` :
 - Initialise les attributs d'instance : `hostname`, `ip_address`.
 - Assigne un `id` unique auto-incrémenté (basé sur `compteur_id`).
 - Initialise un attribut `statut` à `'inactif'`.
- **Méthode** `__repr__(self)` :
 - Doit retourner une chaîne comme `Equipement(id=1, hostname='R1-Paris', ip='192.168.1.1', statut='actif')`.
- **Méthode** `activer(self)` :
 - Change le statut à `'actif'`.
- **Méthode** `desactiver(self)` :

- Change le statut à `'inactif'`.
- **Propriété** `est_actif(self)` :
 - Une propriété en lecture seule qui retourne `True` si le statut est `'actif'`, `False` sinon.

2. Classe `GestionnaireParc` (`gestionnaire.py`)

- **Constructeur** `__init__(self, nom_parc)` :
 - Initialise `nom_parc` et un attribut `equipements` qui est une liste vide.
- **Méthode** `ajouter_equipement(self, equipement)` :
 - Ajoute une instance de `EquipementReseau` à la liste `equipements`. Vérifiez que l'objet ajouté est bien du bon type (`isinstance`).
- **Méthode** `lister_equipements(self)` :
 - Affiche joliment tous les équipements du parc (utilise leur `__repr__`).
- **Méthode** `rechercher_par_hostname(self, hostname)` :
 - Retourne l'instance de `EquipementReseau` correspondant au `hostname`, ou `None` si non trouvé.
- **Méthode** `statistiques(self)` :
 - Affiche le nombre total d'équipements, et combien sont actifs et inactifs.

Code de départ et tests (`main.py`) :

```
# Mettez les classes dans des fichiers séparés (equipement.py, gestionnaire.py)
# et importez-les ici.
# from equipement import EquipementReseau
# from gestionnaire import GestionnaireParc

# --- Tests à valider ---

# 1. Création des équipements
r1 = EquipementReseau(hostname="R1-Paris", ip_address="192.168.1.1")
s1 = EquipementReseau(hostname="S1-Lille", ip_address="192.168.1.10")
```



```

r2 = EquipementReseau(hostname="R2-Lyon", ip_address="10.0.0.1")

assert r1.id == 0 and s1.id == 1, "Les IDs doivent être uniques et séquentiels."
assert r1.statut == 'inactif', "Le statut initial doit être 'inactif'."

# 2. Activation et test de la propriété
r1.activer()
assert r1.est_actif is True, "La propriété est_actif doit refléter le statut."
print(r1)

# 3. Création du gestionnaire et ajout d'équipements
parc_nord = GestionnaireParc("Parc-Nord-France")
parc_nord.ajouter_equipement(r1)
parc_nord.ajouter_equipement(s1)

# 4. Lister et rechercher
print("\n--- Liste des équipements du parc ---")
parc_nord.lister_equipements()

print("\n--- Recherche de 'S1-Lille' ---")
equipement_trouve = parc_nord.rechercher_par_hostname("S1-Lille")
assert equipement_trouve is s1
print(f"Trouvé : {equipement_trouve}")

# 5. Statistiques
print("\n--- Statistiques du parc ---")
# Active s1 pour les stats
s1.activer()
parc_nord.statistiques() # Devrait afficher : Total: 2, Actifs: 2, Inactifs: 0

s1.desactiver()
print("\n--- Statistiques après désactivation ---")

```

```
parc_nord.statistiques() # Devrait afficher : Total: 2, Act  
ifs: 1, Inactifs: 1  
  
print("\nTests du mini-projet passés avec succès !")
```

equipement.py

gestionnaire.py

main.py